

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

AI Assisted Task Manager

Detailed project brief generated from the project record.

CANDIDATE

Rahul Sharma

DURATION

7 days

TECHNOLOGY STACK

JavaScript

TypeScript

React

Next.js

Node.js

Express.js

Laravel (PHP)

Tailwind CSS

MySQL

MongoDB

JWT

Groq API

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

A small web app where users register, manage personal tasks, and receive AI generated task title suggestions via the Groq API.

CANDIDATE Rahul Sharma	TYPE fullstack	DURATION 7 days
--	--	---

Primary Objectives

- Create a responsive Next.js UI for task CRUD.
- Build a Node/Express API with JWT auth and MySQL storage.
- Integrate Groq API to suggest task titles.
- Write unit tests for core endpoints.
- Document setup and API usage.

Candidate Details

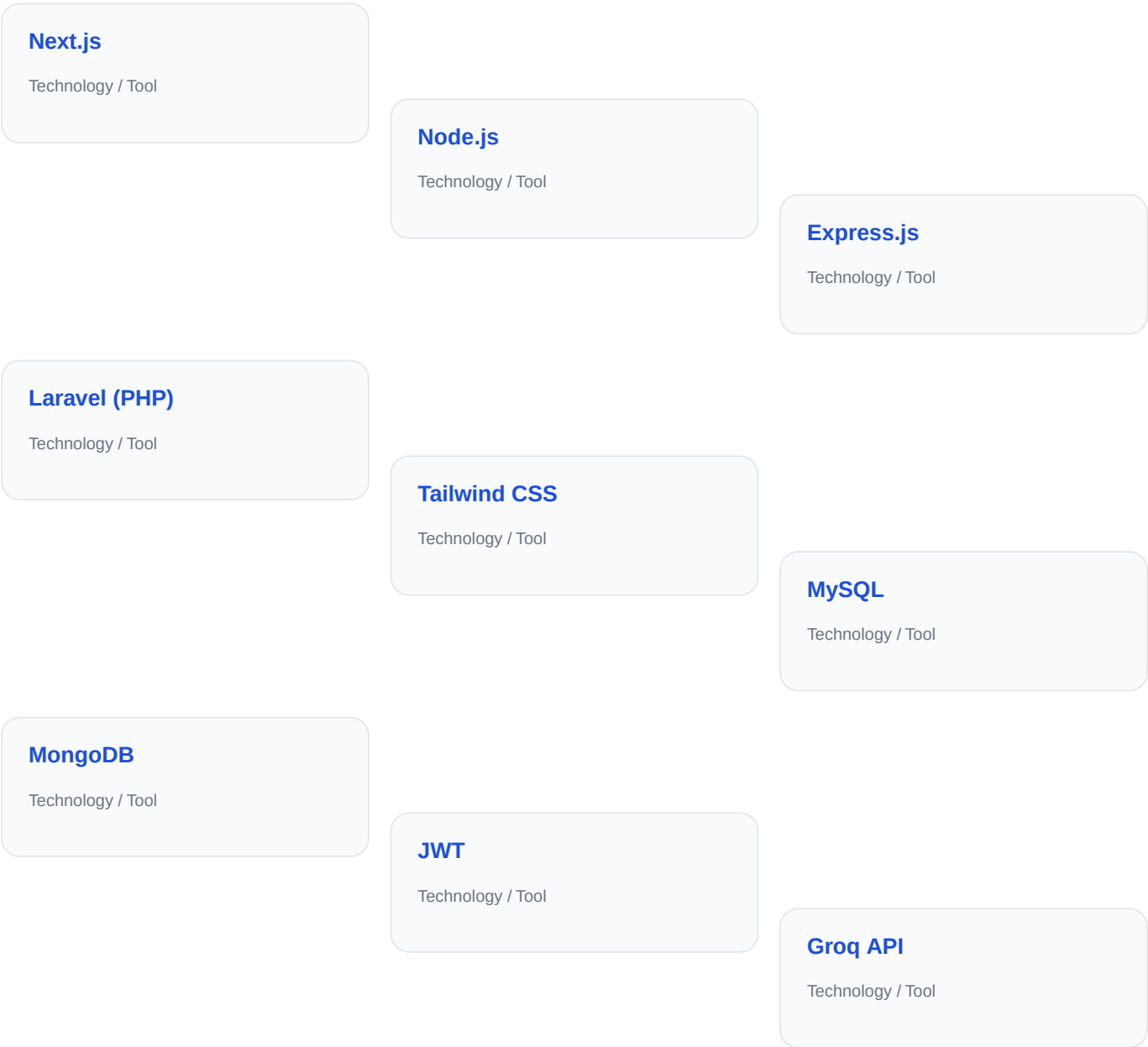
NAME

Rahul Sharma

Technology Stack

Technologies, frameworks and tools expected for implementation.

JavaScript Technology / Tool	TypeScript Technology / Tool	React Technology / Tool
--	--	---



Functional Requirements

- User can register and log in.
- Authenticated users can create, read, update, delete their tasks.
- Each task has title, description, due date, and status.
- User can request an AI generated title suggestion for a task description.
- API returns proper HTTP status codes and validation errors.

Non-Functional Requirements

- Passwords stored with bcrypt hashing.
- JWT tokens expire after 1 hour.
- Input validation on all endpoints.

- Responsive UI for desktop and mobile.
- Basic error logging to console.

Modules & Features

Major functional areas that make up the project.

Authentication

Handles user sign up, login, JWT issuance, and role checks.

- Register endpoint
- Login endpoint
- JWT middleware
- Password hashing

Task Management

CRUD operations for personal tasks.

- Create task
- Read tasks
- Update task
- Delete task

AI Suggestion

Calls Groq API to generate a title from a description.

- Suggestion endpoint
- Prompt construction
- Response parsing

Frontend UI

React/Next.js pages for auth flow and task dashboard.

- Login/Signup pages
- Task list view
- Task editor
- AI suggestion button

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Setup & Foundations

Create Git repo with monorepo layout., Set up Next.js (TS) and Express server., Configure MySQL connection and create Users/Tasks tables., Add ESLint/Prettier.

- Create Git repo with monorepo layout.
- Set up Next.js (TS) and Express server.
- Configure MySQL connection and create Users/Tasks tables.
- Add ESLint/Prettier.

2

Authentication

Build register & login routes., Integrate bcrypt for passwords., Create JWT generation & verification middleware., Write unit tests for auth routes.

- Build register & login routes.
- Integrate bcrypt for passwords.
- Create JWT generation & verification middleware.
- Write unit tests for auth routes.

3

Task CRUD

Create task routes (POST, GET, PUT, DELETE)., Add ownership checks using JWT payload., Validate request bodies., Add basic UI pages for task list and editor.

- Create task routes (POST, GET, PUT, DELETE).
- Add ownership checks using JWT payload.
- Validate request bodies.
- Add basic UI pages for task list and editor.

4

AI Integration

Create /suggest endpoint calling Groq API., Handle API key securely via env variable., Display suggestion in task editor., Write integration test for suggestion flow.

- Create /suggest endpoint calling Groq API.
- Handle API key securely via env variable.
- Display suggestion in task editor.
- Write integration test for suggestion flow.

5

Polish & Delivery

Add end to end tests with Cypress., Write README with setup, env, and API docs., Create simple Dockerfile for backend., Perform final UI responsiveness check.

- Add end to end tests with Cypress.
- Write README with setup, env, and API docs.
- Create simple Dockerfile for backend.
- Perform final UI responsiveness check.

API Design

Key API endpoints and expected responsibilities.

POST

/api/auth/register

Create new user

POST	/api/auth/login	Authenticate and receive JWT
GET	/api/tasks	List user's tasks
POST	/api/tasks	Create a task
PUT	/api/tasks/:id	Update a task
DELETE	/api/tasks/:id	Delete a task
POST	/api/tasks/:id/suggest	Get AI generated title

Evaluation Criteria

Criteria that can be used to assess the completed project.

- Criterion: Correctness | Description: All functional requirements work as specified. | Weight: 30
- Criterion: Code Quality | Description: Readable, modular code with TypeScript typings. | Weight: 20

- Criterion: Security | Description: Password hashing, JWT handling, input validation. | Weight: 15
- Criterion: API Design | Description: RESTful endpoints, proper status codes. | Weight: 15
- Criterion: Testing | Description: Coverage of unit, integration, and e2e tests. | Weight: 10
- Criterion: Documentation & Deployment | Description: Clear README, Dockerfile, and setup instructions. | Weight: 10

Expected Deliverables

Artifacts and outcomes expected at project completion.

- Source code repository
- README with setup & API docs
- Database schema script
- Dockerfile for backend
- Test suite (unit, integration, e2e)
- Demo video or GIF of core flows

Candidate Alignment

Project requirements compared with the candidate information available in the project record.

Rahul already knows JavaScript/TypeScript, React, Next.js, Node/Express, JWT auth, and API design. This 7 day fullstack project leverages those skills while adding a brief foray into AI integration (Groq) and basic Docker deployment, providing measurable outcomes for a junior evaluation.

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.